- Programmation Orientée Objet -

Présentation de Netbeans

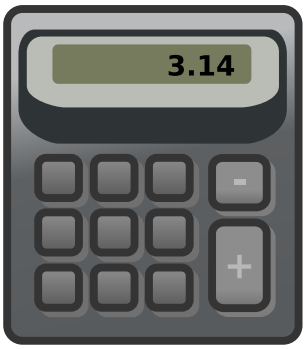
TP0

Intervenante: Georgina ABI SEJAAN

Objectifs du TP

Le but de ce premier TP est de vous familiariser avec **Netbeans**.

Vous allez notamment grâce aux exercices proposés apprendre à :
— **créer un projet** et un source Java,
— utiliser quelques fonctionnalités de l'**éditeur de texte**,
— **compiler et exécuter** un projet,
— **déboguer** un programme.
Enfin, vous réaliserez un **programme complet** de Calculatrice récapitulant tous ces savoirs.



Il vous est conseillé de garder précieusement ce document, il vous sera utile tout au long de l'année.

INSTALLER NETBEANS SELON LES CARACTERISTIQUE DE VITRE ORDINATEUR

Rendu

Teams????

Vous devrez déposer votre travail sur la **plateforme Moodle** du cours, dans le **dépôt TP0Final**

N.B/ VOUS DEVEZ RENDRE UN RAPPORT POUR CHACUNE DES PARTIES DE CE TP.

Le nom du rapport doit contenir votre nom
Exemple : MARTINS_Calculette_TP0

*en collaboration avec MRC

Partie 1 : Affichage d'un message

1. **Créer un dossier de travail LOCAL** à votre nom pour le TP : `C:\MesDocs\TP0_MYNAME`.
2. **Créer un projet nommé `Exo1` :**
   (a) sélectionner ***File -> New Project***
   (b) choisir le type de projet : dans l'onglet ***Categories*** sélectionner ***Java*** et dans l'onglet ***Projects*** sélectionner ***Java Application***
   (c) Dans la fenêtre qui apparait (cf. figure 1 p. 2), saisir le nom du projet : ***Exo1*** et sélectionner le dossier de travail que vous avez crée.

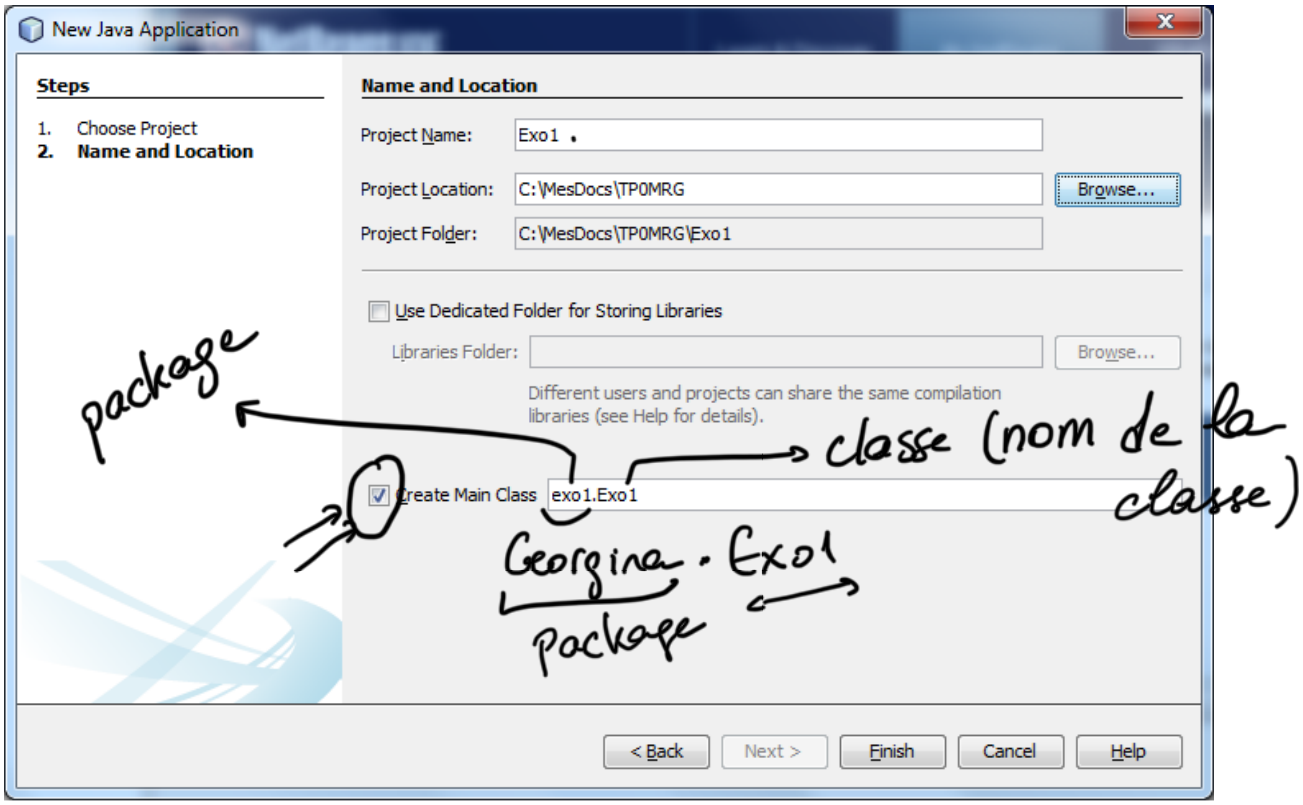


Figure 1 – *Fenêtre "Création d'un nouveau projet Java Application"*

Vous devriez voir apparaître votre fenêtre de travail (figure 2 p. 3) :
— À Gauche l'onglet `Projects` affiche l'arborescence du projet.
— À Droite, la fenêtre de l'éditeur de texte affichant le contenu du nouveau fichier `Exo1.java`
— En Bas, des fenêtres affichant divers messages (compilateur, console, etc.)



L'onglet Projects permet de visualiser la structure logique du projet

— **Source Packages :** comprend les paquetages contenant les fichiers sources du projet,
— **Libraries** comprend les bibliothèques utilisées pour le projet,

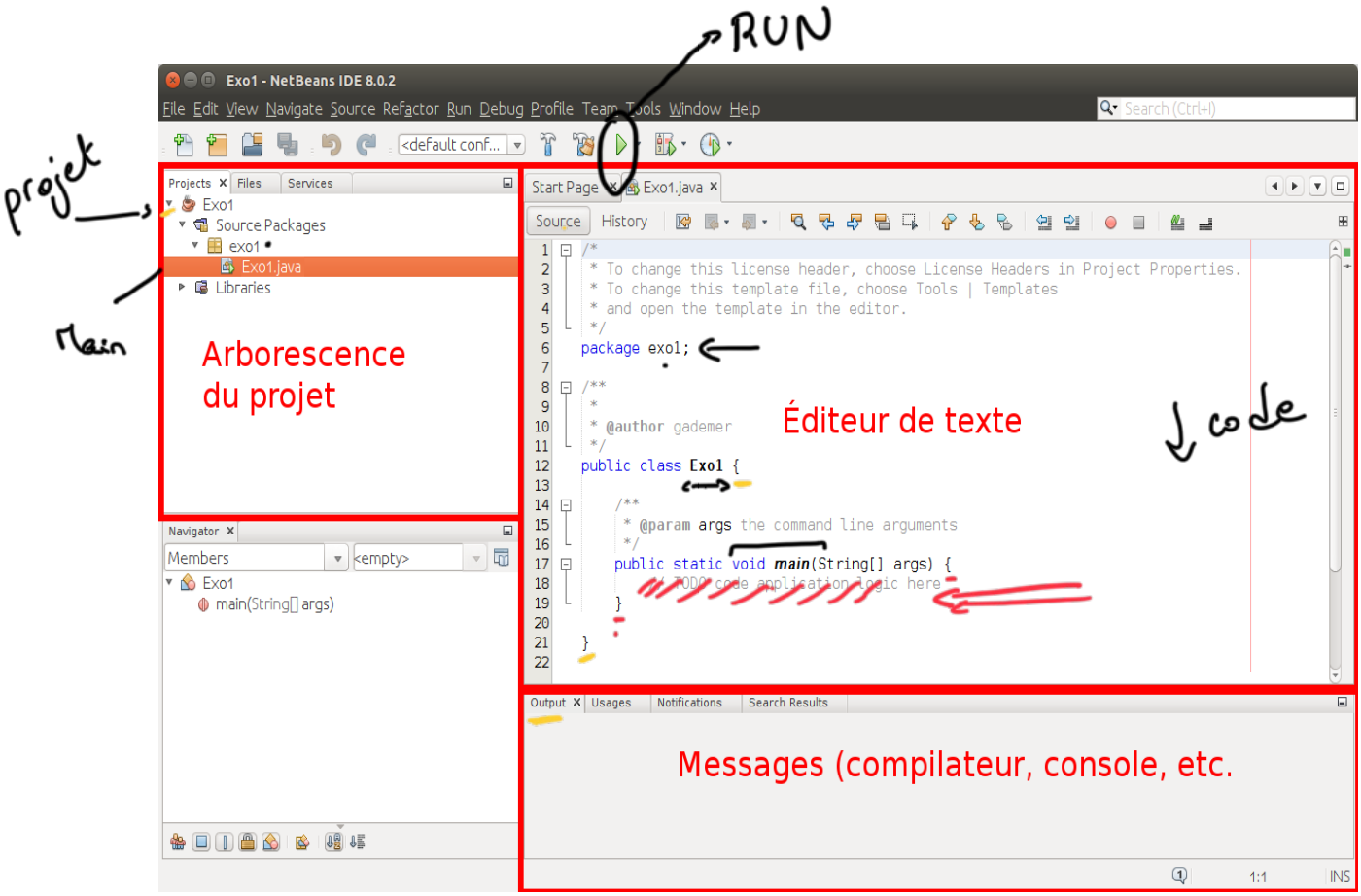


Figure 2 – *Différentes fenêtres de travail de l'interface*

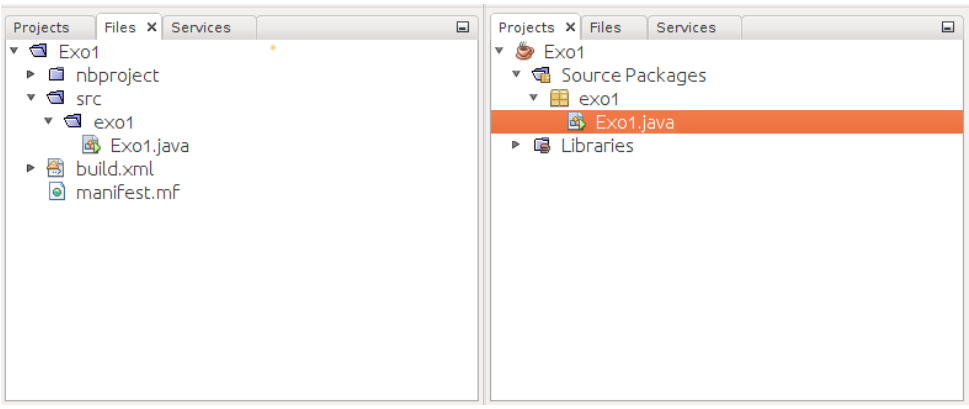


Figure 3 – *Comparaison de l'onglet Files et Projects.*



L'onglet Files permet quant à lui de visualiser la structure physique du projet :

- **build** contient les « bytecode » du projet (fichiers résultant de la compilation),
- **nbproject** contient les fichiers de configuration du projet (utilisés par NetBeans pour savoir comment compiler, exécuter, etc.),
- **src** contient les fichiers sources du projet,
- **build.xml et manifest.mf** sont des fichiers utilisés pour construire le projet et le fichier jar.

C'est ainsi que le peut voir que Netbeans crée un dossier « ***Exo1*** » dans le dossier de travail sélectionné. Ce dossier correspond au projet dans son ensemble (source et fichier Netbeans).

Le sous-dossier `src` contient les fichiers sources de Java selon les règles suivantes :

— un sous-dossier par "package" (regroupement de classe). Ici, Netbeans a crée par défaut un package `exo1`.
— un fichier texte `.java` par classe. Ici, `Exo1.java`

3. **Modifier le fichier principal `Exo1.java` :**

a. Dans l'onglet ***Projects*** cliquer sur ***Exo1-> Source Packages -> exo1 -> Exo1.java***

b. Remplacer les commentaires, en tête du programme, par un cartouche d'identification (noms, groupe, rôle du programme, date).

En Java, les commentaires se notent :
- soit entre /* ... */ (plusieurs lignes)
- soit avec // (une seule ligne).

c. Dans la méthode main, remplacer le commentaire par l'instruction suivante :

```
System.out.println("Bonjour");
```

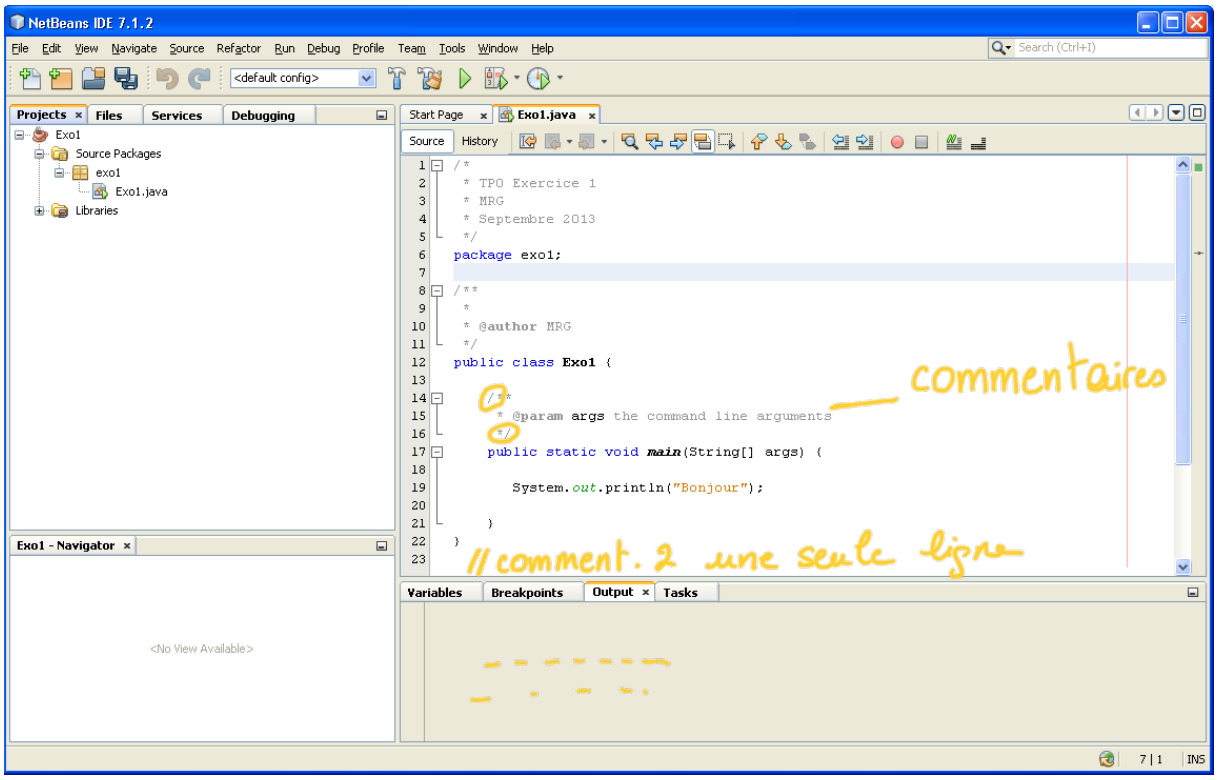
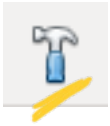


Figure 4 – ***Votre écran après avoir modifié les commentaires et rajouter la ligne de code***

4. **Exécuter le projet :**



Pour compiler/exécuter votre programme Netbeans vous propose 4 icones :



Compiler (uniquement les fichiers modifiés)



Nettoyer tout puis Compiler



Compiler puis Exécuter



Compiler puis Exécuter en mode debug

Sélectionner ***Run -> Build Project*** (ou ***F11*** ou  dans la barre d'outils).
La fenêtre ***Output*** apparait alors dans la zone de message (cf. fig 2 p. 3). Elle montre le résultat de la compilation (`BUILD SUCCESSFULL` normalement).

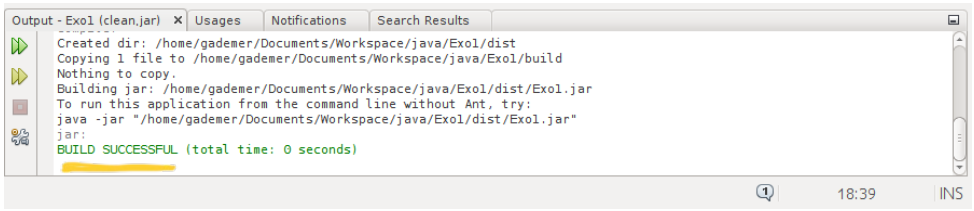


Figure 5 – *Affichage du résultat de la compilation.*

Si vous avez plutôt ce message là :

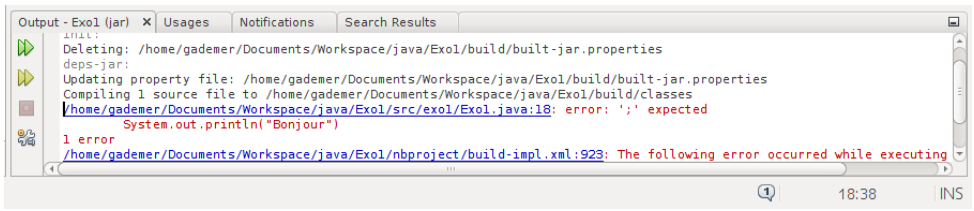


Figure 6 – *Affichage du résultat de la compilation (cas d'une erreur).*

C'est que il y a une erreur dans votre programme ! Le compilateur vous indique la ligne (18) et le type d'erreur (`error ';' expected`).



Compilation à la volée

Netbeans compile votre programme en régulièrement en tache de fond et il vous affiche alors souligné en rouge les lignes qui présente des erreurs (cf. 7 p. 6). Si vous placez le curseur sur le point d'exclamation rouge en début de ligne, vous voyez le message d'erreur.

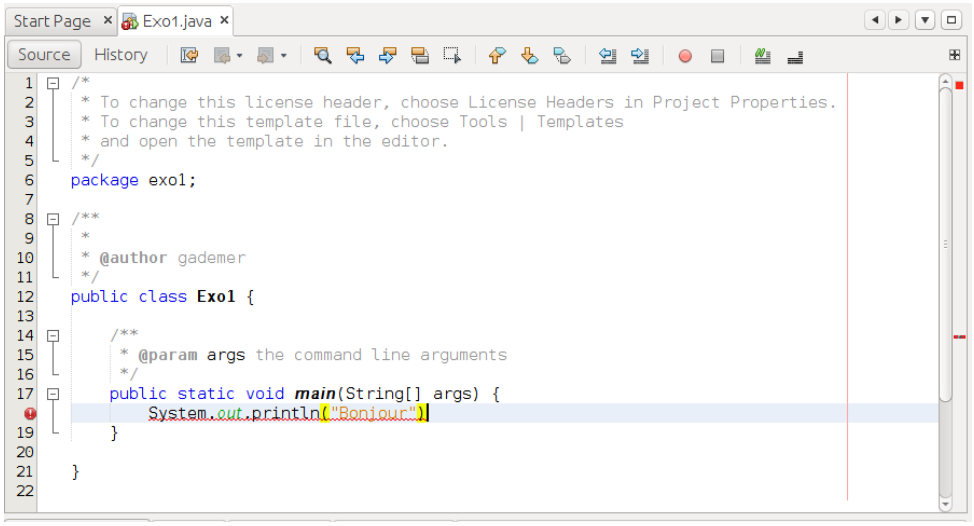


Figure 7 – *Affichage des erreurs à la volée*

Sélectionner ***Run -> Run Project*** (ou ***F6*** ou  dans la barre d'outils).
La fenêtre ***Output*** apparait alors dans la zone de message (cf. fig 2 p. 3). Elle montre le résultat de la compilation (`BUILD SUCCESSFULL`)et de l'exécution du programme (Affichage du message `Bonjour`).

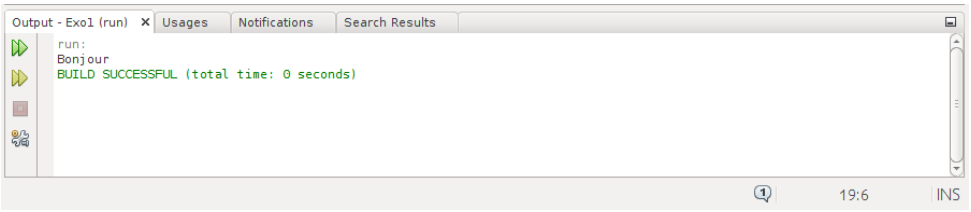


Figure 8 – *Affichage du résultat du programme.*



Astuce n°1 : Autocomplétion

Lorsque vous tapez l'opérateur . (point) ou lors de l'appui des touches `Ctrl+Espace` Netbeans vous fait des proposition de complétion automatique de votre ligne.

Par ex : Si vous tapez `Sytem.out.` Netbeans vous propose les méthodes associées à cet objet (`append`, `print`, etc.)

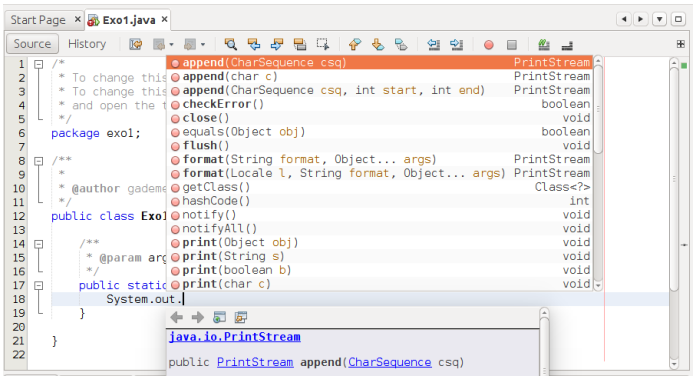


Figure 9 – *Affichage des propositions de complétion*

Astuce n°2 : Suggestions

Netbeans est capable de voir que certaines parties de code sont manquantes ou erronées (par ex : utilisation d'une variable non déclarée) et peut vous proposer de corriger votre code. Pour cela il affiche une petite ampoule jaune au début de la ligne.

Par ex : Si vous tapez `i=20;` Netbeans vous proposera de déclarer la variable `i` ;

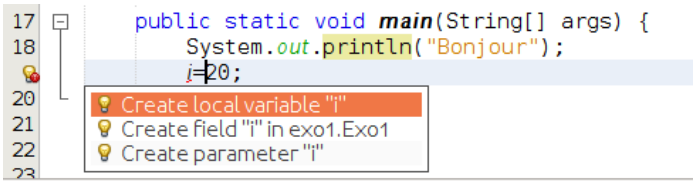


Figure 10 – *Affichage des propositions de correction*

Attention, le système de suggestion n'est ni magique, ni parfait !

Si vous ne comprenez pas la solution qu'il vous propose, il ne vaut mieux pas l'utiliser. L'abus aveugle de suggestion conduit à générer de plus en plus d'erreurs !

Astuce n°3 : Renommer

Lorsque Netbeans voit une variable, il fait automatiquement le lien avec toutes les autres instance de cette variable et il les surlignes à l'écran. Vous pouvez renommer TOUTES les instance de cette variable en une seule fois utilisant `Ctrl+R` ou `Clic Droit > Refactor > Rename`.

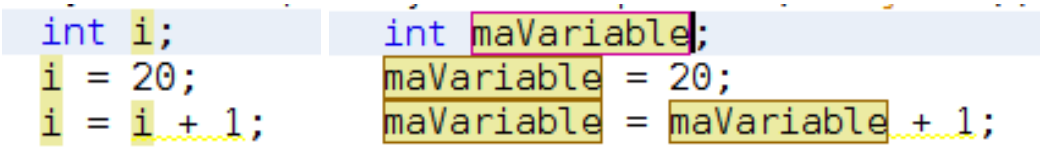


Figure 11 – *À gauche, identification automatique des instances de la variable `i`, à droite renommage de masse.*

5. **Modifiez le programme** : En utilisant l'auto-complétion, ajouter un message d'au revoir.
6. **Personnaliser le programme :**
   Demander son prénom à l'utilisateur.
   Pour lire la chaîne de caractères, utiliser la classe Scanner :
   — recopier les instructions suivantes :

```
String prenom;
Scanner sc;
sc = new Scanner(System.in);
System.out.println("Bonjour, quel est votre prenom ?");
prenom = sc.nextLine();
```

   — cliquer sur l'ampoule apparue devant la déclaration de la variable *sc*, et sélectionner « `Ajouter une importation pour java.util.Scanner` ».
   — Modifier votre programme pour personnaliser le message affiché par le programme.

Partie 2 : Addition d'entiers

1. **Créer un nouveau projet nommé `Exo2`**
2. **Dans le fichier principal,**
   (a) Faire le cartouche d'identification.
   (b) Ajouter dans la méthode main, le code suivant sans le modifier :

```
public static void main(String[] args) {

   //Declaration des variables
   int nb; // nombre d'entiers a additionner
   int result; // resultat
   int ind; //indice
   nb=5;
   result=0;

   // Addition des nb premiers entiers
   ind=1;
   while (ind <= nb) {
      result=result+ind;
   }

   // Affichage du resultat
   System.out.println();
   System.out.println("La somme des "+ nb + "entiers est: "+result);
}
```

3. Visualiser les numéros de ligne : sélectionner ***View -> Show Line Numbers.***
4. Exécuter le projet : le programme ne s'arrête pas. Pour en arrêter l'exécution,
— soit sélectionner ***Run -> Stop Build/Run***
— soit cliquer sur le bouton rouge dans la fenêtre ***Output*** .

5. Utiliser le débogueur :

a. 1ère possibilité :
— Placer le curseur de la souris sur la ligne « *ind=1 ;* ».
— Exécuter le programme jusqu'au curseur en sélectionnant ***Debug -> Run to Cursor*** (ou ***F4***).

2ème possibilité :
— Cliquer avec la souris dans la marge de gauche sur le numéro de la ligne « *ind=1 ;* » (la ligne est alors surlignée en rose).
— Exécuter le programme jusqu'à cette ligne en sélectionnant ***Debug -> Debug Project***.

b. Lorsque la ligne est surlignée en vert, sélectionner l'onglet ***Variables*** (à côté de l'onglet ***Output***). Il permet de visualiser la valeur des variables utilisées dans le programme.

c. Exécuter le programme en pas à pas : sélectionner
soit ***Debug -> Step Over*** (ou ***F8***)
soit ***Debug -> Step Into*** (ou ***F7***)

d. Trouver le problème et corriger le code.

6. Modifier la méthode main, pour saisir le nombre d'entiers à additionner :
déclarer une variable *nb* de type entier, et utiliser la classe Scanner :

```
int nb;
//nb=5;
Scanner sc = new Scanner(System.in);
System.out.println("\n Entrer le nombre :");
nb=sc.nextInt(); // On demande a sc de donner le prochain entier
```

Partie 3 : Calculette

EXERCICE 1

 Commencez par créer un **nouveau projet** `Calculator` et modifiez le cartouche d'identification.

EXERCICE 2

 Ajoutez les lignes de code dans la méthode `main` qui permettent d'afficher le texte suivant :

```
Please enter the operator:
1) add
2) substract
3) multiply
4) divide
5) modulo
```

EXERCICE 3

 Ajoutez les lignes de code dans la méthode `main` qui permettent de récupérer une valeur entière saisie par l'utilisateur et stockez la dans une variable `operateur`.



Rappelez vous que la classe `Scanner` possède des méthodes dédiées : `nextLine()`, `nextInt()`, `nextFloat()`,...

EXERCICE 4

 Ajoutez les lignes de code dans la méthode `main` qui permettent de demander à l'utilisateur une première valeur en affichant un message puis en récupérant sa saisie dans une variable `operande1`.

EXERCICE 5

 Faites de même pour `operande2`.

EXERCICE 6 (*Le minimum*)

 En fonction des valeurs choisies pour l'opérateur, calculez le résultat de l'opération puis affichez-le à l'écran.



Embranchements

En Java, comme en C, on peut utiliser `if` / `else` ou `switch`.

Exemple d'exécution :

```
Please enter the operator:
1) add
2) substract
3) multiply
4) divide
5) modulo
3
Please enter the first number:
```

```
5
Please enter the second number:
9
The result is : 45
```

EXERCICE 7 (*Le petit plus*)



Lorsque l'opérateur choisi n'est pas entre 1 et 5 inclus, affichez un message d'erreur à la place.

EXERCICE 8 (*Encore mieux*)



Faites le test de l'opérateur erroné AVANT de demander les opérandes.



Interrompre le programme

`System.exit(0);` permet d'interrompre le déroulement du programme en cas d'erreur de saisie par exemple.